umber of Connected Components in an Undirecte

Graphs | LeetCode: https://leetcode.com/problems/number-of-connected-compo

Problem

- N nodes and a list of undirected edges. Return the number of connected components.

Intuition

- Start with n components; every successful union of two different components reduces the count by one.

Brute Force

- DFS/BFS from each unvisited node, counting starts: O(V + E), also fine.

Optimal Approach

- Components = n.
- For each edge, if union(u, v) merges two roots, components--.

Worked Example

```
n = 5, edges [[0,1],[1,2],[3,4]].

start 5
[0,1] merge -> 4
[1,2] merge -> 3
[3,4] merge -> 2
answer 2
```

Java Solution

```java
public class NumberOfConnectedComponents {
    public int countComponents(int n, int[][] edges) {
        int[] parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        int comps = n;
        for (int[] e : edges) {
            int a = find(parent, e[0]), b = find(parent, e[1]);
            if (a != b) { parent[a] = b; comps--; }
        }
        return comps;
    }

    private int find(int[] p, int x) {
        while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
        return x;
    }
}
```

Complexity

- Time: O(E alpha(n)). Space: O(n).

Pitfalls

- Duplicate or already-connected edges must not decrement the count.